

ISTANBUL KÜLTÜR UNIVERSITY



Capstone Project
ZeroCollab: P2P Project Management and Assessment Tool

Submitted By

Songül Nur Öztürk 2100004806
Özlem Nur Bora 1900003285
Nisanur Karataş 2100004398

Project Advisor
Yusuf Altunel

Department of Computer Engineering
İstanbul Kültür University

2026 Spring Term

June
2026



ISTANBUL KÜLTÜR UNIVERSITY

Capstone Project
ZeroCollab: P2P Project Management and Assessment Tool

Submitted By

Songül Nur Öztürk 2100004806
Özlem Nur Bora 1900003285
Nisanur Karataş 2100004398

Advisor
Yusuf Altunel

THE EXAMINATION COMMITTEE

Jury Member	Signature
1. ……..	……..
2. ……..	……..
3. ……..	……..

ABSTRACT

This report presents ZeroCollab, an offline-first agile project management tool that we developed for our capstone project. The main idea behind it was simple: most teams rely on tools like Jira or Trello that become difficult to use when the internet connection is unavailable. We wanted to build something that still works when the connection is unstable, so we combined peer-to-peer communication, local browser storage, and cloud backup in a single hybrid setup.

The platform covers the usual agile workflow pieces: sprint planning, a Kanban board, a product backlog, a roadmap, dashboard analytics, messaging, a calendar, and a basic video meeting room. On the technical side, peer-to-peer connections run on PeerJS over WebRTC, local data sits in IndexedDB through the idb library, and Supabase handles authentication and the cloud backup layer.

This hybrid setup lets teams keep working when the network is unstable. During development we spent a lot of time on synchronization issues across the three storage layers, on protecting newer local data from being overwritten by stale snapshots after page reloads, and on keeping multi-user editing stable. The final result is a working prototype that shows decentralized agile tooling can be a practical alternative to fully cloud-dependent platforms.

ÖZET

Bu raporda, bitirme projemiz kapsamında geliştirdiğimiz ZeroCollab adlı çevrimdışı öncelikli çevik proje yönetim aracı tanıtılmaktadır. Projenin çıkış noktası, geleneksel proje yönetim araçlarının internet bağlantısı kesildiğinde kullanılamaz hale gelmesi sorunudur. Bu sorunu aşmak için eşler arası iletişim, yerel depolama ve bulut yedeklemeyi tek bir hibrit yapı içinde birleştirdik.

Sistemde sprint planlama, Kanban tabanlı görev takibi, ürün birikim listesi yönetimi, yol haritası, panel analitikleri, mesajlaşma, takvim ve prototip düzeyinde bir görüntülü görüşme odası bulunmaktadır. Teknik olarak eşler arası bağlantılar PeerJS üzerinden WebRTC ile sağlanır, yerel veriler idb kütüphanesi aracılığıyla IndexedDB içinde tutulur, kimlik doğrulama ve bulut yedeklemesi ise Supabase üzerinden yapılır.

Bu yapı sayesinde takım üyeleri bağlantı sorunlarının yaşandığı durumlarda da çalışmaya devam edebiliyor. Geliştirme sürecinin büyük bir bölümü üç depolama katmanı arasındaki senkronizasyon problemlerine, sayfa yenileme sonrası eski verilerin yenisini ezme sorununa ve çok kullanıcılı düzenlemenin tutarlı kalmasına ayrıldı. Ortaya çıkan sonuç, merkezi olmayan çevik araçların tamamen buluta bağımlı platformlara pratik bir alternatif olabileceğini gösteren çalışan bir prototiptir.

ACKNOWLEDGEMENTS

We would like to thank the Department of Computer Engineering at Istanbul Kültür University for giving us the chance to work on this capstone project.

We are especially grateful to our advisor, Yusuf Altunel, for his guidance and patience throughout the process. His feedback on our backlog and sprint hierarchy pushed us to rework an early version that did not really fit Scrum, and the final structure is much cleaner because of it. This work also makes use of the results of the Ontology Based Software Engineering and Zero Time Software Development studies carried out by Yusuf Altunel within the Department and at LC Waikiki, which informed the ontology-driven design of our evaluation and configuration layers.

We are grateful to the open-source communities behind the technologies we used, including React, Vite, Supabase, PeerJS, IndexedDB (idb), Tailwind CSS, and Recharts.

Finally, we thank our families and friends for their patience and moral support during the long hours of development and writing.

TABLE OF CONTENTS

ABSTRACT I
ÖZET II
ACKNOWLEDGEMENTS III
TABLE OF CONTENTS IV
LIST OF TABLES V
LIST OF FIGURES VI
SYMBOLS & ABBREVIATIONS VII
1. INTRODUCTION 1
    1.1 Problem Statement 1
    1.2 Project Purpose 1
    1.3 Project Scope 2
    1.4 Objectives and Success Criteria 2
    1.5 Report Outline 3
2. RELATED WORK 4
    2.1 Existing Systems 4
    2.2 Overall Problems of Existing Systems 4
    2.3 Comparison Between Existing and Proposed Method 4
3. METHODOLOGY 6
    3.1 Requirement Analysis 6
    3.2 Design 7
    3.3 Implementation 12
    3.4 Testing 14
4. EXPERIMENTAL RESULTS 15
5. DISCUSSION 19
6. CONCLUSIONS 21
REFERENCES 22
APPENDIX A: Project File Structure 23
APPENDIX B: Test Summary 24
APPENDIX C: Team Contribution Summary 25

LIST OF TABLES

Table 1.1: Objectives and Success Criteria of ZeroCollab 2
Table 2.1: Comparison of Existing Project Management Tools and ZeroCollab 4
Table 3.1: Functional Requirements of ZeroCollab 6
Table 3.2: Non-Functional Requirements of ZeroCollab 7
Table 3.3: Technology Stack Summary 12
Table 4.1: Synchronization Test Results 15
Table 4.2: Module Completion Status 15

LIST OF FIGURES

Figure 3.1: Use Case Diagram – ZeroCollab System 8
Figure 3.2: System Architecture Diagram (Hybrid P2P + Cloud) 9
Figure 3.3: Hybrid Synchronization Flow Diagram 10
Figure 3.4: Sprint Planning Activity Diagram 11
Figure 3.5: Supabase Database Schema (ER Diagram) 12
Figure 4.1: Product Backlog Screenshot 16
Figure 4.2: Product Roadmap Screenshot 17
Figure 4.3: Sprint Board Screenshot 17
Figure 4.4: Dashboard Screenshot 18

SYMBOLS & ABBREVIATIONS

API	Application Programming Interface
CRDT	Conflict-Free Replicated Data Type
DB	Database
HTTP	Hypertext Transfer Protocol
IDB / IndexedDB	Indexed Database API – Browser-based local storage
JS	JavaScript
JSX	JavaScript XML – React component syntax extension
P2P	Peer-to-Peer
PWA	Progressive Web Application
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
WebRTC	Web Real-Time Communication

1. INTRODUCTION

Most software teams today rely on centralized project management tools to coordinate sprints, tasks, and communication. These tools work well when everything is online, but they share one weakness: they need a steady internet connection and the vendor's servers to keep running. When the network gets slow or a service goes down, work effectively stops and team data can become unreachable.

ZeroCollab was our answer to this problem. We combined peer-to-peer (P2P) communication, offline-first local storage, and an optional cloud sync layer so the platform stays usable even when the connection is unreliable. It is a Single Page Application built with React 19 (Meta Open Source, 2024) and Vite. PeerJS (PeerJS Contributors, 2024) and the Web Real-Time Communication (WebRTC) standard (World Wide Web Consortium, 2021) handle the real-time peer layer, IndexedDB (Mozilla Developer Network, 2024) through the idb library (Archibald, 2022) keeps data on the client side, and Supabase (Supabase, 2024) takes care of authentication and the cloud backup.

1.1 PROBLEM STATEMENT

Traditional agile project management platforms such as Jira (Atlassian, 2024), Trello, and Monday.com rely entirely on centralized cloud infrastructure. This design introduces several recurring problems:

I. Network dependency: All collaborative actions require a stable internet connection. Any disruption halts collaboration entirely.

II. Synchronization conflicts: In distributed teams, concurrent edits to the same items can generate conflicts that centralized servers may not resolve predictably.

III. Data ownership concerns: Teams uploading sensitive project data to third-party servers may face regulatory, privacy, or intellectual property risks.

IV. Cost and lock-in: Commercial platforms charge per seat and make data migration difficult, creating vendor dependency.

1.2 PROJECT PURPOSE

The purpose of this project is to design, implement, and evaluate a decentralized agile project management platform supporting full sprint lifecycle management, real-time team collaboration, and offline-first operation.

a) WebRTC-based peer-to-peer connections can reliably synchronize agile work items between team members in real time.

b) IndexedDB can serve as a robust local persistence layer, ensuring data availability without internet access.

c) Supabase cloud storage can be integrated as a non-blocking fallback synchronization layer.

d) A modular React Single Page Application (SPA) architecture can support complex agile workflows including sprint planning, backlog management, roadmap visualization, messaging, and video meetings.

1.3 PROJECT SCOPE

1. User authentication and workspace management via Supabase
2. Product Backlog management with full Create, Read, Update, and Delete (CRUD) operations, Scrum-compliant hierarchy, and P2P synchronization
3. Product Roadmap with Now/Next/Later prioritization buckets
4. Sprint Planning with backlog item assignment and capacity management
5. Sprint Board (Kanban) with drag-and-drop task management
6. Dashboard analytics: task status distribution, workload by member, weekly activity charts
7. Messaging: direct messages and project-scoped channels
8. Meeting room with a basic WebRTC video calling implementation
9. Calendar view for task and meeting scheduling
10. Settings: workspace configuration, color themes, data export
11. Offline support via Progressive Web App (PWA) capabilities and IndexedDB persistence

The current version of ZeroCollab focuses mainly on the web platform and offline-first collaborative workflows. In addition to the core synchronization system, the project also includes an AI-assisted evaluation module with ontology-driven rubric structures and a token-based feature management layer using Hedera Testnet (Hedera, 2024). Some areas, such as wallet onboarding, semantic evaluation automation, and broader cross-platform support, are still being improved as the project evolves.

1.4 OBJECTIVES AND SUCCESS CRITERIA OF THE PROJECT

Table 1.1 summarizes the main objectives of the project together with the success criterion defined for each objective.

Table 1.1: Objectives and Success Criteria of ZeroCollab

Objective	Success Criterion
Real-time P2P synchronization	Backlog and sprint items sync between two peers within 2 seconds of an update
Offline-first persistence	All data remains accessible and editable when disconnected from the internet
Cloud backup integration	Supabase successfully stores and retrieves project data for authenticated users
Sprint lifecycle management	Users can create, populate, and close sprints with backlog item assignments
Stale-state protection	Merge-based guards prevent older snapshots from overwriting newer data after page reload
Core module functionality	All major modules (Backlog, Roadmap, Sprint, Kanban, Dashboard, Chat, Meetings, Calendar, Token Economy) operational with core functionality implemented

1.5 REPORT OUTLINE

Section 2 surveys existing project management tools and examines their limitations. Section 3 describes the methodology covering requirement analysis, system design, implementation, and testing. Section 4 presents experimental results. Section 5 discusses findings and lessons learned. Section 6 concludes and outlines future work.

2. RELATED WORK

A review of existing agile project management tools and supporting technologies was conducted prior to the design phase.

2.1 EXISTING SYSTEMS

Jira (Atlassian): Most widely adopted agile tool, offering comprehensive Scrum and Kanban support. Fully cloud-dependent with no offline capability. Enterprise pricing makes it inaccessible to smaller teams.

Trello: Simple Kanban board for lightweight task management. Lacks sprint planning, roadmap, and reporting features. Fully cloud-dependent.

Monday.com: Flexible board and timeline views with automation. Fully centralized SaaS with no P2P layer and no offline support.

Microsoft Teams / Azure DevOps: Integrates project tracking with communication. Heavyweight, requires organizational Microsoft accounts, and provides no decentralized synchronization.

Linear: Developer-oriented tool with clean UX. Cloud-first approach with no P2P capabilities; offline mode limited to read-only cached content.

Yjs / Automerge (Academic CRDTs): Research frameworks exploring conflict-free replicated data types (Kleppmann et al., 2019). Academically valuable but low-level; no complete agile PM solution.

2.2 OVERALL PROBLEMS OF EXISTING SYSTEMS

1. Complete internet dependency: All reviewed systems require a continuous, stable internet connection.
2. Centralized failure point: Server-side outages directly prevent all team activity.
3. No peer-to-peer real-time collaboration: None of the tools use WebRTC or similar P2P protocols.
4. Limited data sovereignty: All project data stored on third-party servers.
5. High cost at scale: Per-seat pricing models make commercial tools prohibitive for growing teams.
6. No hybrid persistence model: None combine local IndexedDB storage with cloud backup.

2.3 COMPARISON BETWEEN EXISTING AND PROPOSED METHOD

Table 2.1 compares ZeroCollab with the existing centralized agile tools across the capabilities discussed in this section.

Table 2.1: Comparison of Existing Project Management Tools and ZeroCollab

Feature	Jira	Trello	Monday	Linear	ZeroCollab (Ours)
Sprint Planning	Yes	No	Partial	Yes	**Yes (full)**
Kanban Board	Yes	Yes	Yes	Yes	**Yes (full)**
Product Backlog	Yes	No	Partial	Yes	**Yes (full)**

Roadmap	Yes	No	Yes	Yes	**Yes (full)**
P2P Real-time Sync	No	No	No	No	**Yes (WebRTC)**
Offline Support	No	No	No	Limited	**Yes (IndexedDB)**
Video Meetings	No	No	No	No	**Prototype (WebRTC)**
Open Source / Free	No	No	No	Partial	**Yes**
Local Data Storage	No	No	No	No	**Yes (IDB)**
Cloud Backup	Yes	Yes	Yes	Yes	**Yes (Supabase)**

ZeroCollab combines P2P synchronization, offline-first storage, and built-in collaboration features in a single platform. The meeting module is still an early implementation rather than a polished feature.

3. METHODOLOGY

ZeroCollab was developed following an agile software development methodology, planned and executed in iterative sprint cycles with backlog items continuously refined based on team review and advisor feedback.

An important process note: the initial agile workflow hierarchy used in the early sprints mixed Epics, Features, User Stories, and Tasks in a flat structure that did not align with Scrum principles. After advisor feedback, the backlog and sprint hierarchy was redesigned to follow a proper Scrum structure with clear level separation.

3.1 REQUIREMENT ANALYSIS

3.1.1 Functional Requirements

Table 3.1 lists the functional requirements identified for ZeroCollab together with a short description of each.

Table 3.1: Functional Requirements of ZeroCollab

ID	Requirement	Description
FR-01	**User Authentication**	Register, log in, and log out using Supabase email/password authentication.
FR-02	**Workspace Management**	Create projects, invite team members, and manage workspace settings.
FR-03	**Product Backlog**	Create, edit, delete, and reorder backlog items with Scrum-compliant type classification (Task, User Story, Feature, Epic).
FR-04	**Sprint Planning**	Create sprints, set dates, assign backlog items, and close sprints.
FR-05	**Sprint Board (Kanban)**	Manage tasks through columns: To Do, In Progress, In Review, Done.
FR-06	**Product Roadmap**	Create roadmap items in Now/Next/Later prioritization buckets.
FR-07	**Dashboard Analytics**	Display task status pie charts, member workload bars, and weekly activity line charts.
FR-08	**P2P Synchronization**	Synchronize backlog, roadmap, sprint, and task data between peers in real time via PeerJS/WebRTC.
FR-09	**Messaging**	Send direct messages and project-scoped channel messages.
FR-10	**Meetings**	Schedule meetings and conduct WebRTC video calls (initial implementation, not all network setups tested).
FR-11	**Calendar**	View tasks and meetings in a monthly calendar layout.
FR-12	**Offline Persistence**	All data accessible and editable offline using IndexedDB.
FR-13	**Data Export**	Export project data as JSON or ZIP archives.
FR-14	**Activity Log**	Record and display an audit trail of user and system actions within the project, with filtering, search, and export.
FR-15	**AI Evaluation**	Evaluate uploaded project deliverables against ontology-defined quality dimensions using a Large Language Model (LLM), producing scored rubric feedback.

3.1.2 Non-Functional Requirements

Table 3.2 presents the non-functional requirements of the system, grouped by quality category.

Table 3.2: Non-Functional Requirements of ZeroCollab

Category	Requirement
Performance	P2P sync events must complete within 2 seconds under normal network conditions.
Reliability	IndexedDB persistence must ensure zero data loss during page reloads and offline sessions.
Usability	All major workflows must be completable within 3 user interactions from the main navigation.
Scalability	The modular architecture must support addition of new modules without refactoring existing ones.
Security	All cloud operations must be authenticated via Supabase Row Level Security policies.
Compatibility	The application must function correctly on modern Chromium-based and Firefox browsers.
Maintainability	Components must be organized by domain (pages/, components/, features/, utils/) for clarity.

3.2 DESIGN

The design of ZeroCollab is described through a set of complementary models. This section first captures the system behaviour from the user's perspective with a use-case diagram, then presents the overall system architecture, the hybrid synchronization flow, the sprint-planning activity flow, and finally the database schema. Together these models show how the functional and non-functional requirements identified above are realized in the structure of the system.

3.2.1 Use Case Diagram

Figure 3.1 illustrates the primary use cases of ZeroCollab from the perspective of two user roles: the Project Owner and Team Member. Stick-figure actors appear on the left; oval use cases are shown inside the system boundary. Blue ovals are shared by both roles; orange ovals are exclusive to the Project Owner.

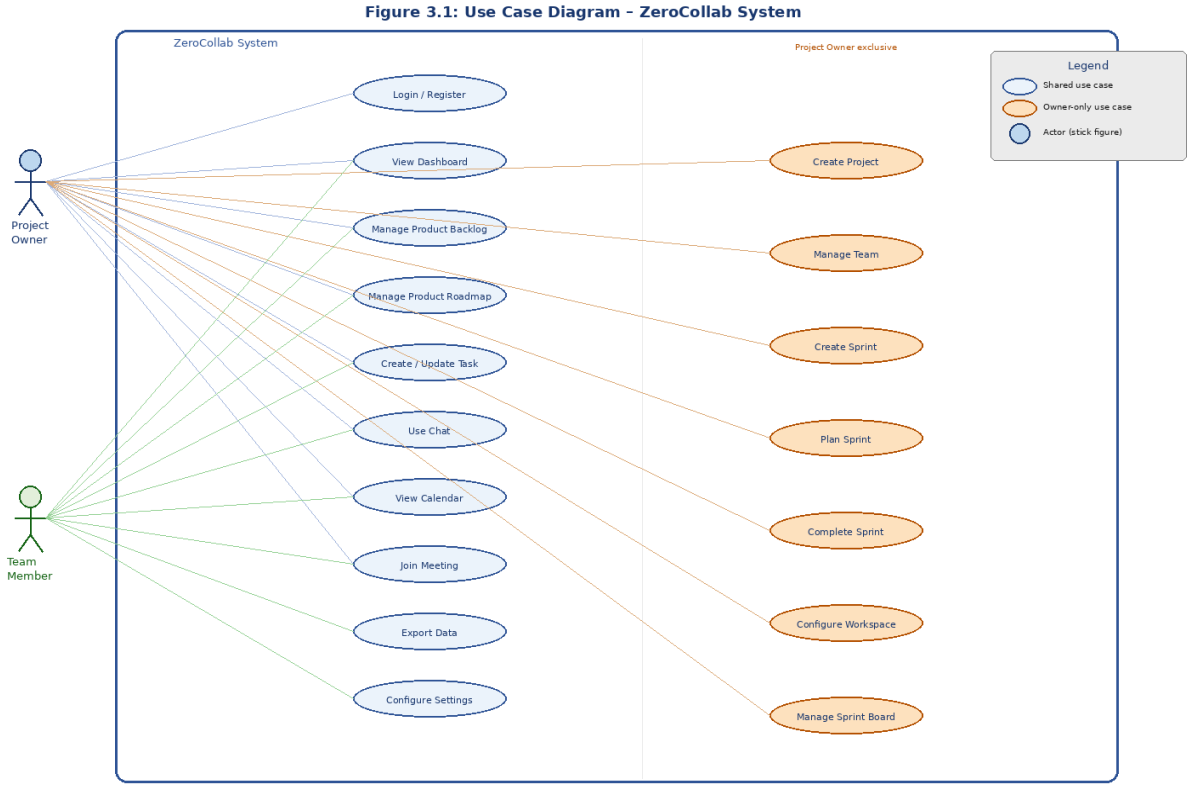


Figure 3.1: Use Case Diagram – ZeroCollab System

3.2.2 System Architecture Diagram

Figure 3.2 illustrates the five-layer hybrid architecture. Layer 1 is the React SPA frontend; Layer 2 is the application logic; Layer 3 is the P2P communication layer using PeerJS/WebRTC; Layer 4 is local persistence via IndexedDB; and Layer 5 is durable cloud persistence via Supabase. The AI-assisted evaluation module, which uses ontology-defined rubric structures, runs within the application logic layer and consumes these same persistence layers rather than forming a separate tier.

Figure 3.2: System Architecture Diagram - Hybrid P2P + Cloud

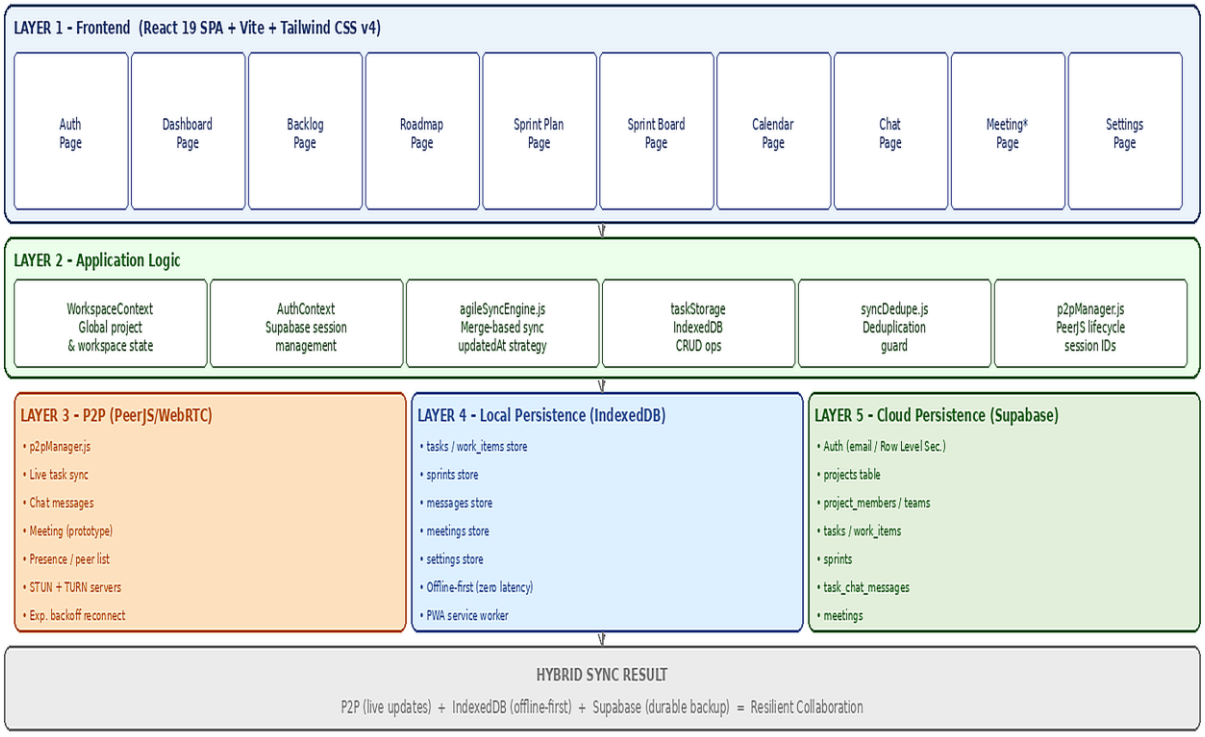


Figure 3.2: System Architecture Diagram (Hybrid P2P + Cloud)

Layer 1 – Frontend (React SPA): All user interaction occurs through a React 19 SPA built with Vite. Components are organized into pages/, components/, features/, and utils/. Tailwind CSS v4 provides utility-first styling.

Layer 2 – Application Logic: WorkspaceContext and AuthContext manage global state and authentication. agileSyncEngine.js coordinates synchronization using merge-based guards. taskStorage handles all IndexedDB CRUD operations.

Layer 3 – P2P (PeerJS/WebRTC): p2pManager.js manages PeerJS connection lifecycle. Each user receives a stable base peer ID stored in IndexedDB plus a session-specific UUID suffix. Exponential backoff handles reconnection. Session Traversal Utilities for NAT (STUN) and Traversal Using Relays around NAT (TURN) servers enable Network Address Translation (NAT) traversal.

Layer 4 – Local Persistence (IndexedDB): IndexedDB via the idb library serves as the primary local storage. All CRUD operations execute against local storage first, with zero network latency. Each module maintains its own object store.

Layer 5 – Cloud Persistence (Supabase): Supabase provides PostgreSQL-backed cloud storage with Row Level Security. Cloud sync runs asynchronously as a non-blocking background operation.

3.2.3 Hybrid Synchronization Flow Diagram

Figure 3.3 shows how data flows through the three persistence layers. When User A makes a change, it is saved locally to IndexedDB first, then broadcast via P2P to connected peers, then backed up to Supabase. A late-joining User C receives data through cloud hydration and local IDB restore.

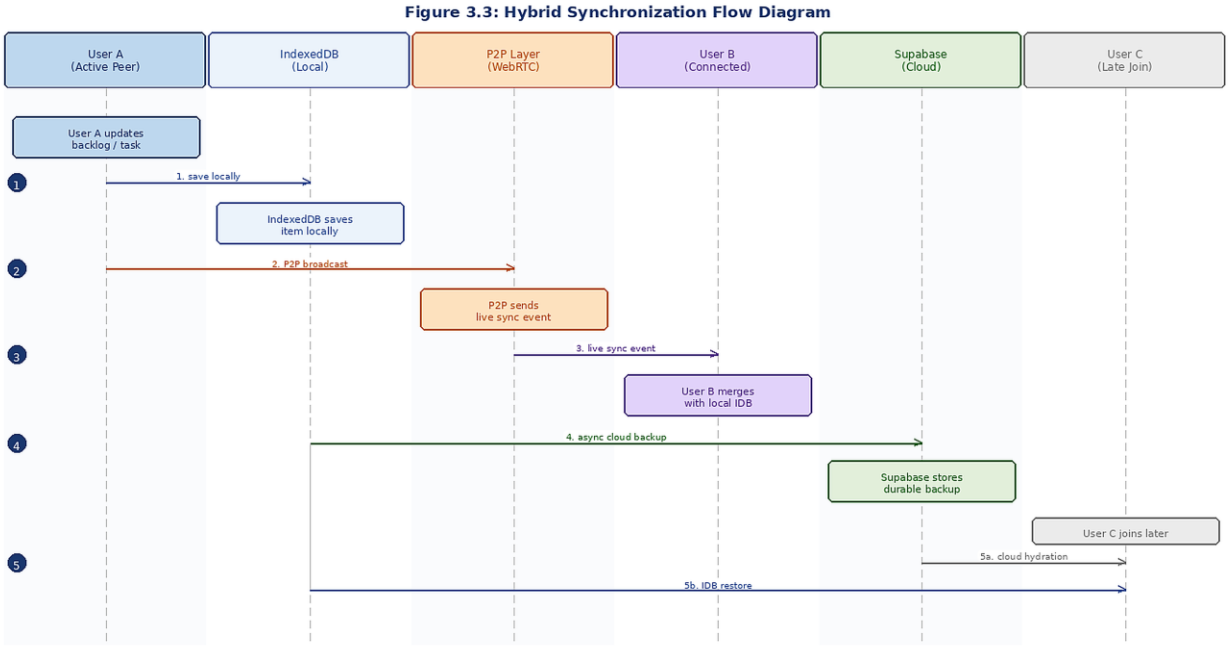


Figure 3.3: Hybrid Synchronization Flow Diagram

3.2.4 Sprint Planning Activity Diagram

Figure 3.4 shows the sprint planning workflow. Importantly, Epics and Features are NOT directly added to sprints. The sprint accepts only Ready User Stories and Independent Tasks, reflecting the Scrum-compliant hierarchy adopted after advisor feedback.

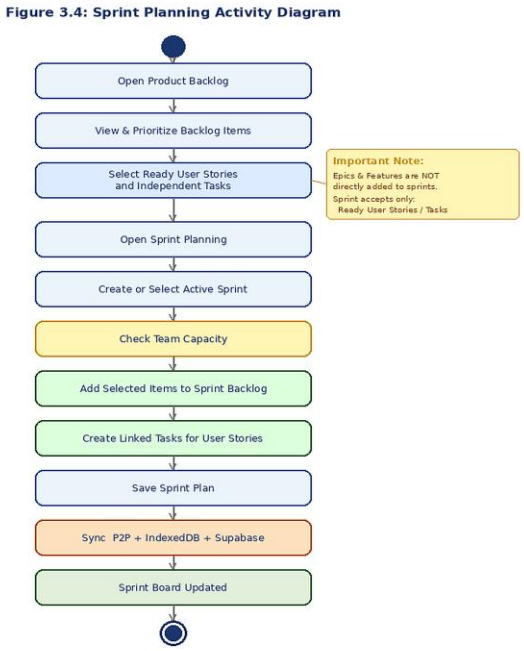


Figure 3.4: Sprint Planning Activity Diagram

3.2.5 Database Schema (ER Diagram)

Figure 3.5 presents the Supabase database schema. The schema defines nine primary tables with foreign-key relationships supporting all project management, team, task, sprint, messaging, and meeting operations.

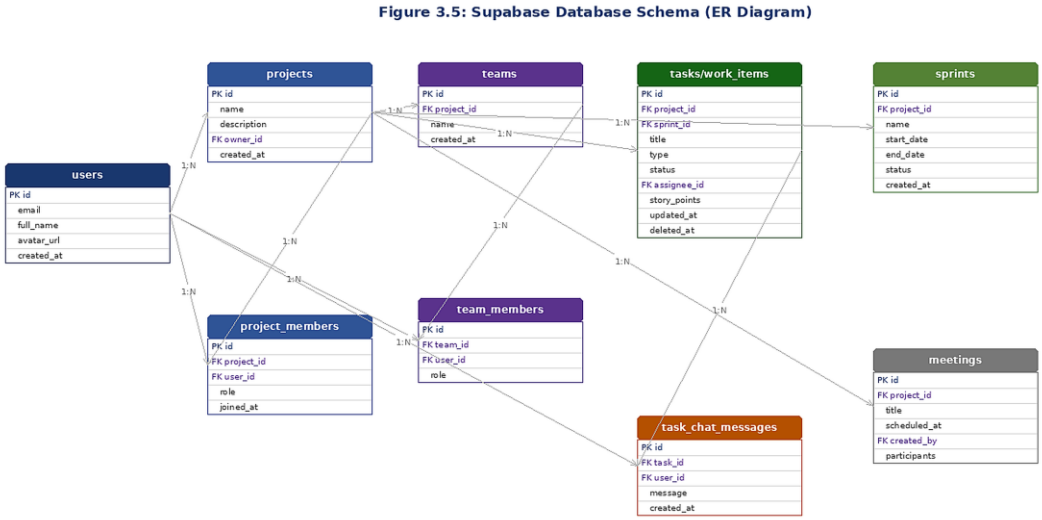


Figure 3.5: Supabase Database Schema (ER Diagram)

3.3 IMPLEMENTATION

The implementation spans 17 major source files across pages/, components/, features/, and utils/.

Table 3.3 summarizes the technology stack used in the implementation, listing each technology together with its role in the system.

Table 3.3: Technology Stack Summary

Category	Technology	Version / Role
Frontend Framework	React	v19.2.0 – Component-based UI
Build Tool	Vite	v7.3.1 – Fast development and production builds
CSS Framework	Tailwind CSS	v4.2.1 – Utility-first styling
Routing	React Router DOM	v7.13.1 – Client-side SPA routing
P2P Communication	PeerJS	v1.5.5 – WebRTC abstraction layer
Local Storage	idb (IndexedDB)	v8.0.3 – Browser-based persistent storage
Cloud Backend	Supabase	v2.103.0 – Auth and PostgreSQL storage

Charts / Analytics	Recharts (Recharts Team, 2024)	v3.8.0 – Dashboard data visualization
PWA Support	vite-plugin-pwa	v1.2.0 – Service worker and caching
Zip Export	JSZip	v3.10.1 – Data export packaging
Testing	Vitest	v3.2.4 – Unit and integration tests
Test Utilities	Testing Library	v16.3.2 – React component testing

P2P Manager (p2pManager.js): Creates a unique session peer ID by combining a stable base ID (stored in IndexedDB) with a random 8-character UUID suffix. Connection state transitions between OFFLINE, CONNECTING, and ONLINE. Exponential backoff with a maximum 10-second delay handles reconnection. ICE servers include Google STUN servers and OpenRelay TURN servers for NAT traversal.

Agile Sync Engine (agileSyncEngine.js): Provides normalization functions for all agile work item types. Items carry stable UUIDs and updatedAt timestamps. The merge strategy is timestamp-based last-write-wins: merge operations prefer the item with the newer timestamp. Soft deletion (tombstoning) propagates deletions across peers. CRDT-based synchronization is planned for future versions.

P2P Sync Challenges Encountered: During development, several specific synchronization problems were identified and addressed:

1. Stale reload overwrite: Page reloads triggered full cloud re-fetch that could overwrite newer local P2P state.
2. Navigation-triggered state resets: Navigating between pages caused unexpected state rehydration from stale snapshots.
3. Peer reconnection inconsistency: Reconnecting peers sometimes received incomplete state due to race conditions in the sync sequence.
4. IndexedDB snapshot conflicts: Concurrent writes from multiple tabs could produce inconsistent local state.

These problems were resolved through the merge-based guard system in agileSyncEngine.js, the zc-tasks-changed event pattern, and the stable base/session peer ID architecture.

An experimental token economy module using Hedera Testnet was also explored during development. A small ZeroCollab (ZC) token balance is read through the Mirror Node Representational State Transfer (REST) Application Programming Interface (API) and used to gate Free, Plus, and Premium feature tiers on the client. The layer is treated as an experimental capability rather than a production-ready integration; wallet onboarding is still manual and the Mirror Node can lag a few seconds behind the ledger.

Ontology-Driven Configuration (Zero-Time Approach): A distinguishing aspect of ZeroCollab is that several behaviours are defined in ontology files rather than hard-coded in the application. Three Terse RDF Triple Language (TTL) ontology files are kept under public/ontology/: kanban-schema.ttl defines the Kanban board columns and the allowed task state transitions, token-tiers.ttl defines the Free, Plus, and Premium feature tiers

together with their quota limits, and zerocollab-evaluation.ttl defines the quality dimensions and scoring rubric used by the evaluation module. Dedicated loader modules (ontologyLoader, kanbanSchemaLoader, tokenTierLoader, and roleOntology) read these files at runtime. Because the rules live in the ontology, the board structure, the feature tiers, or the evaluation rubric can be changed by editing a single .ttl file, without modifying or recompiling the application code. This reflects the Zero-Time configuration idea, in which the configurable part of the system is adjusted at the knowledge level instead of through code changes.

LLM-Assisted Evaluation Module (evaluationUtils.js): The evaluation feature uses a Large Language Model (LLM) to assess uploaded project deliverables against the rubric defined in zerocollab-evaluation.ttl. The model automatically determines which of the twelve quality dimensions — such as functionality, peer-to-peer behaviour, ontology usage, code quality, testing, and security — apply to the submitted material, and assigns each applicable dimension a score out of ten. The dimension scores are then combined using ontology-defined weights, so that core dimensions such as functionality, P2P, ontology, and blockchain count more heavily than supporting ones, producing a single weighted score that is mapped to a letter grade through configurable thresholds. Because both the dimensions and their weights are defined in the ontology, the assessment criteria can be tuned without changing the evaluation code.

3.4 TESTING

Unit Tests (definitionOfReady.test.js): Verify that the system correctly identifies whether a backlog item meets Definition of Ready criteria. All 5 test cases pass.

Integration Tests (taskStorage.test.js): Verify that tasks are correctly persisted to and retrieved from IndexedDB, that merge operations correctly prefer newer timestamps, and that soft-deleted items are correctly filtered. All 11 test cases pass.

Manual Testing: Multi-user synchronization tested by opening the application in two browser windows simultaneously. Offline scenarios tested by disabling network access in browser developer tools. Beyond these representative cases, the project includes a broader automated suite of 456 test cases across 36 files covering agile logic, chat, calendar, token economy, ontology loading, and UI utilities (see Appendix B).

4. EXPERIMENTAL RESULTS

This section presents the results of functional testing, synchronization testing, and module completion assessment. All tests were conducted using Google Chrome v124 on a Windows 11 development machine.

4.1 SYNCHRONIZATION TEST RESULTS

Table 4.1 reports the synchronization test results, showing the average propagation time and data-consistency outcome for each test scenario.

Table 4.1: Synchronization Test Results

Test Scenario	Avg. Propagation	Data Consistency	Result
Backlog item creation sync	< 800 ms	Consistent across runs	**PASS**
Backlog item deletion (tombstone)	< 900 ms	No issues seen	**PASS**
Sprint task status update	< 750 ms	Consistent across runs	**PASS**
Roadmap milestone creation	< 850 ms	Worked as expected	**PASS**
Merge conflict (concurrent edit)	< 1200 ms	Consistent except rapid concurrent edits*	**PASS**
Post-reload stale snapshot protection	N/A (local)	No overwrite detected in tests	**PASS**
Offline CRUD then reconnect sync	< 2000 ms	No problems observed	**PASS**

* Inconsistency in the merge conflict scenario occurs when two peers simultaneously update the same field within a very short time window (under 100 ms). Flagged for future CRDT-based resolution.

4.2 MODULE COMPLETION ASSESSMENT

Table 4.2 summarizes the completion status of each module, indicating which modules are fully implemented and which remain at a basic level.

Table 4.2: Module Completion Status

Module	Status	Notes
Product Backlog	**Implemented**	Full CRUD, P2P sync, Supabase integration, Scrum hierarchy
Product Roadmap	**Working**	Now/Next/Later buckets, milestones, P2P sync, cloud storage
Sprint Planning	**Functional**	Sprint creation, backlog assignment, close sprint flow
Sprint Board (Kanban)	**Implemented**	Drag-drop columns, task detail modal, basic burndown
Dashboard Analytics	**Working**	Status pie, workload bar, weekly activity line charts

Messaging	**Functional**	Direct messages and project channels operational
Meetings	**Prototype Level**	WebRTC video calls tested; reconnect stability issues noted
Calendar	**Implemented**	Monthly view with task and meeting indicators
Settings	**Working**	Theme, workspace config, data export functional
PWA / Offline	**Functional**	Service worker, offline banner, full IDB persistence
Evaluation	**Implemented**	AI-assisted assessment using ontology-defined rubrics, project-scoped shared history, scope-aware file analysis
Activity Log	**Functional**	Audit trail of project actions with category filtering, search, and export
Token Economics	**Functional**	Hedera Testnet integration, ZC token balance tier gating (Free/Plus/Premium), Mirror Node balance lookup; treated as experimental capability

4.3 APPLICATION SCREENSHOTS

Figures 4.1 through 4.4 show the four main modules of ZeroCollab as rendered in Google Chrome.

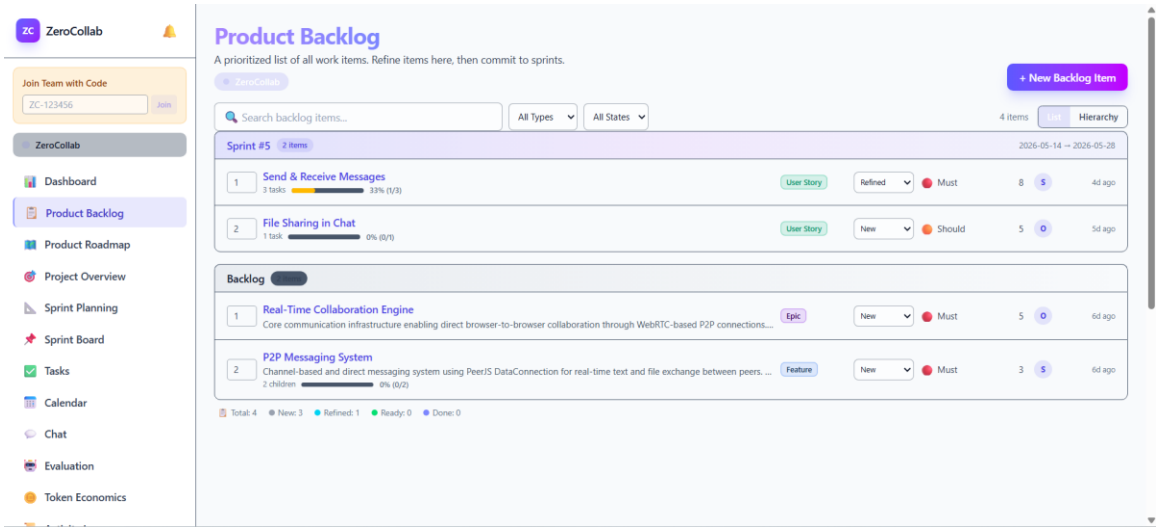


Figure 4.1: Product Backlog – backlog hierarchy with Epic, Feature, and User Story items organized across sprint and backlog sections.

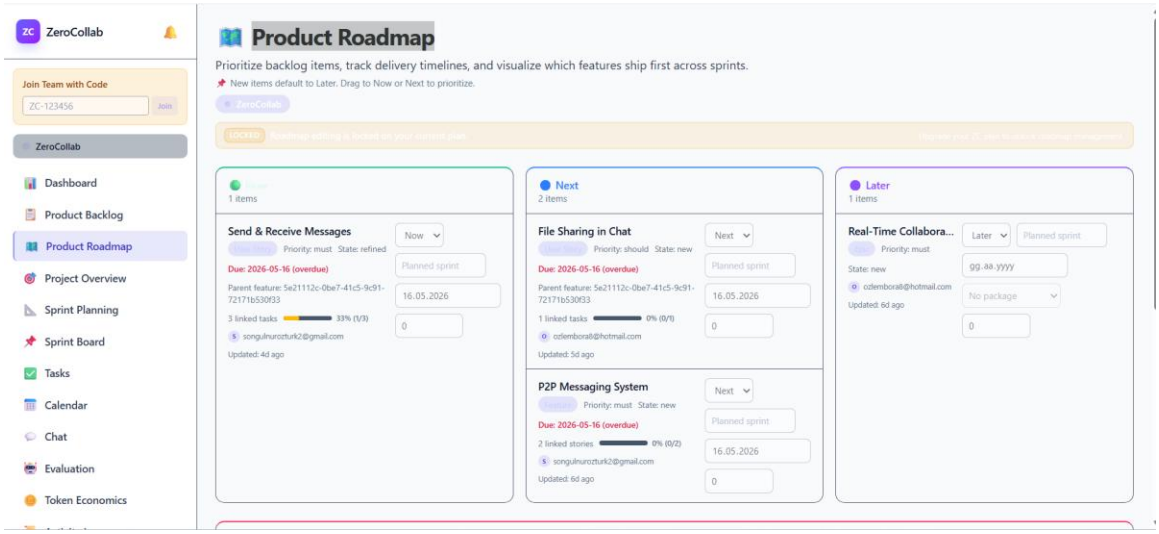


Figure 4.2: Product Roadmap – Now / Next / Later prioritization buckets.

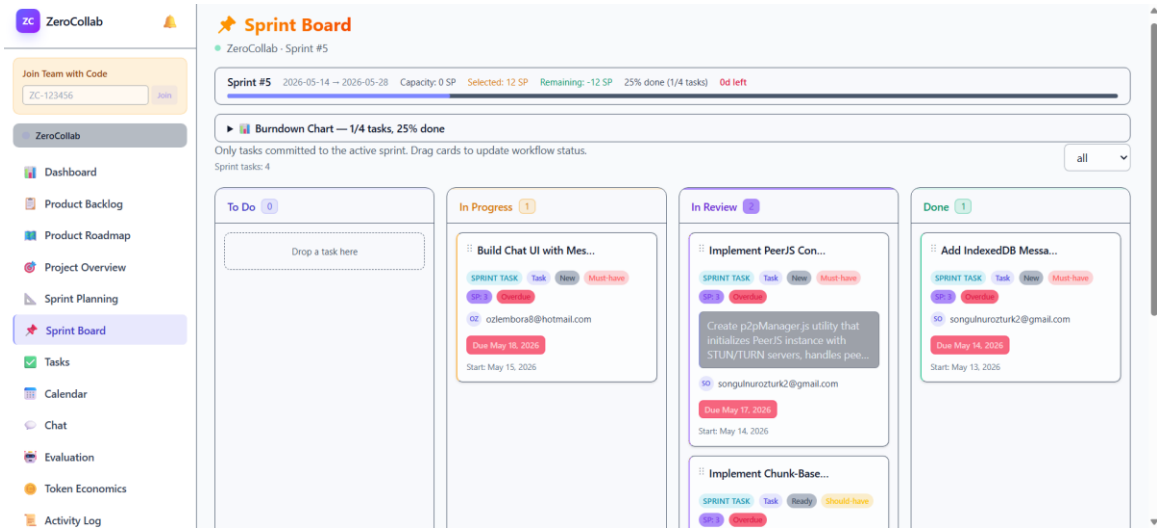


Figure 4.3: Sprint Board (Kanban) – To Do / In Progress / In Review / Done columns with drag-and-drop support.

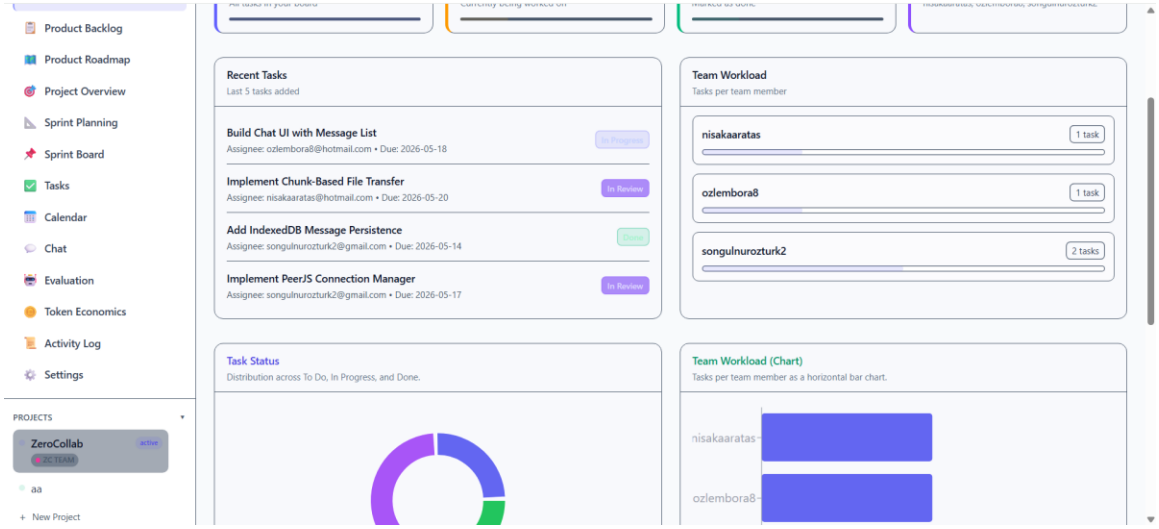


Figure 4.4: Dashboard – task status charts, team workload panels, recent tasks, and project statistics.

5. DISCUSSION

When we started ZeroCollab the goal was to see whether decentralized agile project management could actually work as a real alternative to cloud-only tools. Looking at the results, we feel the project showed that this is possible within the scope of a student capstone.

Development Process and Project Management: ZeroCollab was developed by a three-person team following an iterative, Scrum-inspired process. Work was organized into sprints, and the project itself was tracked using the same Epic / Feature / User Story / Task backlog hierarchy that the tool implements. Responsibilities were divided by feature area — broadly, multi-project workspace and authentication; sprint, Kanban, and task workflows; and calendar, settings, and the token-economy layer — while the AI-assisted evaluation module, the ontology layer, the peer-to-peer (P2P) synchronization engine, and IndexedDB persistence were built as shared work, as summarized in Appendix C. The advisor reviewed our backlog and sprint hierarchy during the project, which led us to redesign an early structure that did not properly follow Scrum. The main risks we faced were technical rather than organizational: the stale-state synchronization problem, the reliability of P2P connections across different Network Address Translation (NAT) configurations, and the timestamp-based merge limitation discussed below. We managed these risks by isolating the synchronization logic into a dedicated engine, continuously testing offline and multi-tab scenarios, and documenting the known merge limitation openly rather than hiding it.

Stale-State Synchronization: The hardest bug we ran into was the stale-state problem on the Backlog and Roadmap pages. Whenever a user reloaded the page, the app would pull the latest snapshot from Supabase, but in some cases that snapshot was actually older than what had already been synced peer-to-peer in the session. The result was that fresh edits got overwritten by older cloud data. We fixed this by adding merge-based guards inside agileSyncEngine that compare updatedAt timestamps and keep the newer record on both sides.

Agile Workflow Redesign: A non-technical issue that took us a while to notice was the agile hierarchy itself. In our first version we threw Epics, Features, User Stories, and Tasks into the same flat list, which felt fine at first but turned out to be confusing once the backlog grew. After our advisor pointed this out, we rebuilt the structure properly so Epics break down into Features, Features into Stories, and Stories into Tasks. The change required some refactoring but made the rest of the work much easier.

P2P Connection Reliability: WebRTC over PeerJS worked well on the local network but became unreliable when we tested it from networks behind strict NAT. Adding OpenRelay TURN servers helped a lot, although it did not solve every case. The meeting module gave us the most trouble: participants would sometimes show up twice in the list, and reconnecting after a dropped call was inconsistent once more than two peers were involved. That is the main reason we kept it as a basic implementation instead of presenting it as a finished feature.

IndexedDB as Primary Store: Treating IndexedDB as the main storage layer rather than just a cache turned out to be one of the better decisions we made. Every read and write hits local storage first, so the UI feels instant, and the Supabase sync runs quietly in the

background without blocking anything. We initially tried the opposite (cloud-first with IDB as a fallback cache), and the UI lag during normal use was enough for us to switch the model.

Limitations: There is one known limitation in our merge logic. Because we use a timestamp-based last-write-wins rule, two users editing the exact same field within roughly 100 ms of each other can end up with one of the edits silently dropped. In day-to-day use this almost never happens, but it is a real correctness gap. The proper fix is to move to a CRDT-based approach, which we left as future work.

6. CONCLUSIONS

ZeroCollab is our attempt at building an agile project management tool that does not depend on a single server staying online. The final version combines WebRTC peer-to-peer communication, IndexedDB for offline storage, and Supabase for cloud backup, all wired together through a custom sync engine. We finished the main features we set out to build — Product Backlog, Roadmap, Sprint Planning, Sprint Board, Dashboard, Messaging, Calendar, Settings, an AI-assisted evaluation module that scores deliverables against ontology-defined rubrics, an activity log, and a token economy layer on Hedera Testnet — and the Meeting Room is working as a basic implementation.

Our main question was whether P2P sync could realistically support agile workflows, and the results suggest that it is workable for typical usage within the scope of a student project. Across the scenarios we measured, updates propagated between two peers in under two seconds, and changes made while offline correctly merged with cloud state after reconnecting. There are still rough edges, but the core idea holds up.

Several directions for future work have been identified:

1. CRDT-based conflict resolution to replace timestamp-based last-write-wins merging for sub-100 ms conflict scenarios
2. Extending the AI-assisted evaluation module with sprint-review assistance and richer server-side validation, building on the ontology-defined rubric system already in place
3. Full production-grade WebRTC meeting support, resolving the reconnection and participant stability issues encountered
4. Advanced analytics module with detailed burndown chart visualization and velocity tracking
5. Extending the ontology layer, currently used for evaluation rubrics and configuration, toward task assignment and dependency detection
6. Mobile-optimized Progressive Web App with push notification support

Overall, working on ZeroCollab showed us that a small team can put together a functional agile collaboration tool using freely available web technologies, even when the design moves away from the standard client-server model.

REFERENCES

Atlassian. (2024). Jira software documentation. Retrieved from https://support.atlassian.com/jira-software-cloud/

Archibald, J. (2022). idb: IndexedDB with usability [Computer software]. Retrieved from https://github.com/jakearchibald/idb

Hedera. (2024). Hedera Token Service documentation. Retrieved from https://docs.hedera.com/hedera/sdks-and-apis/sdks/token-service

Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. In Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019) (pp. 154–178). Association for Computing Machinery.

Meta Open Source. (2024). React documentation. Retrieved from https://react.dev

Mozilla Developer Network. (2024). IndexedDB API. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API

PeerJS Contributors. (2024). PeerJS documentation. Retrieved from https://peerjs.com/docs

Recharts Team. (2024). Recharts documentation. Retrieved from https://recharts.org/en-US

Supabase. (2024). Supabase documentation. Retrieved from https://supabase.com/docs

World Wide Web Consortium. (2021). *WebRTC 1.0: Real-time communication between browsers* (W3C Recommendation). Retrieved from https://www.w3.org/TR/webrtc/

APPENDIX A: PROJECT FILE STRUCTURE

The following directory structure represents the organization of the ZeroCollab source code:

```
ZeroCollab/
  src/
    pages/
      AuthPage.jsx              (User login/registration)
      DashboardPage.jsx         (Analytics dashboard)
      ProductBacklogPage.jsx    (Backlog management)
      ProductRoadmapPage.jsx    (Roadmap management)
      SprintPlanningPage.jsx    (Sprint management)
      SprintBoardPage.jsx       (Kanban sprint board)
      CalendarPage.jsx          (Task/meeting calendar)
      ChatPage.jsx              (Messaging shell)
      MeetingRoomPage.jsx       (Video meetings – prototype)
      SettingsPage.jsx          (Settings shell)
    components/
      backlog/ | kanban/ | sprint/ | chat/ | meetings/ | calendar/ |
settings/ | ui/
    features/
      agile/sync/
        agileSyncEngine.js      (Central P2P sync engine)
        syncDedupe.js           (Deduplication guard)
    utils/
      p2pManager.js             (PeerJS/WebRTC manager)
      AuthContext.jsx           (Authentication context)
      WorkspaceContext.jsx      (Project/workspace state)
      LanguageContext.jsx       (Internationalization)
  package.json
  vite.config.js
```

APPENDIX B: TEST SUMMARY

The automated test suite was executed using Vitest:

1. Agile and Kanban logic – definitionOfReady, taskStorage, taskVisibility, taskDoneAt, kanbanBoardFilters, kanbanSchemaLoader, sprintBurndown, sprintPlanningHierarchy, sprintProgressUtils
2. Chat and messaging – chatConstants, chatDmIds, chatStorage, taskChatUtils, taskChatRemoteStorage
3. Calendar and scheduling – calendarDateUtils, customCalendarStorage, conflictDetection, meetingStorage, timezoneUtils
4. Token economy and ontology – tokenTiers, tokenTierLoader, quotaService, featureGateService, ontologyLoader, evaluationUtils, roleOntology
5. Dashboard, UI, PWA and other utilities – dashboardTaskUtils, dashboardVisibleTasks, uiComponents, OfflineBanner, pwaInstall, authValidation, notificationService, translations, taskRemoteStorage

The suite spans 36 test files containing 456 automated test cases across unit and integration levels, all passing as of the final submission date.

APPENDIX C: TEAM CONTRIBUTION SUMMARY

Team Member	Student ID	Primary Contributions
Songül Nur Öztürk	2100004806	Calendar integration, settings management, PWA/offline support, Token Economy and Hedera integration, automated testing with Vitest, Definition of Ready logic. Shared contributions: AI-assisted evaluation module, ontology-defined rubric structures, P2P synchronization engine, IndexedDB persistence, Supabase synchronization, collaborative workflow implementation, debugging and UI integration.
Özlem Nur Bora	1900003285	Product Backlog, Product Roadmap, backlog hierarchy design, system architecture design, Supabase database schema and migration design, multi-project workspace context management, user authentication flow, Burndown Chart integration, task management workflows and roadmap milestones. Shared contributions: AI-assisted evaluation module, ontology-defined rubric structures, P2P synchronization engine, IndexedDB persistence, Supabase synchronization, collaborative workflow implementation, debugging and UI integration.
Nisanur Karataş	2100004398	Sprint Planning, Sprint Board (Kanban), task workflow management, Dashboard analytics, messaging and task chat systems, meetings module, activity log system and notification workflows. Shared contributions: AI-assisted evaluation module, ontology-defined rubric structures, P2P synchronization engine, IndexedDB persistence, Supabase synchronization, collaborative workflow implementation, debugging and UI integration.